Cross-compiling user-space libraries and applications



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Integrating user-space libraries and applications

- One of the advantages of embedded Linux is the wide range of third-party libraries and applications that one can leverage in its product
- There's much more than U-Boot, Linux and BusyBox that we can reuse from the open-source world
- Networking, graphics, multimedia, crypto, language interpreters, and more.
- Each of those additional software components needs to be cross-compiled and installed for our target
- Including all their dependencies
  - Which can be quite complex as open-source encourages code reuse



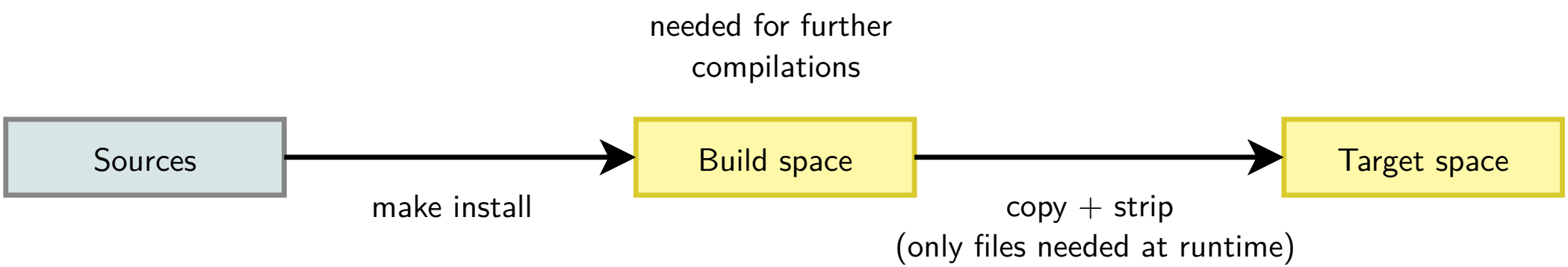
Concept of build system

- Each open-source software project comes with its own set of scripts/files to control its configuration/compilation: its *build system*
  - Detect if system requirements/dependencies are met
  - Compile all source files, to generate applications/libraries, as well as documentation
  - Installs build products
- Most common build systems:
  - Hand-written *Makefiles*
  - *Autotools*: *autoconf*, *automake*, *libtool*
    https://en.wikipedia.org/wiki/GNU_Autotools
  - *CMake*
    https://cmake.org/
  - *Meson*
    https://mesonbuild.com/
  - Language specific build systems for Python, Perl, Go, Rust, NodeJS, etc.



Target and staging spaces

- When manually cross-compiling software, we will distinguish two “copies” of the root filesystem
  1. The target root filesystem, which ends up on our embedded hardware, which contains only what is needed for *runtime*
  2. The staging space, which has a similar layout, but contains a lot more files than the *target* root filesystem: headers, static libraries, documentation, binaries with debugging symbols. Contains what’s needed for *building* code.
- Indeed, we want the root filesystem on the target to be as minimal as possible.





Cross-compiling with hand-written Makefiles

- There is no general rule, as each project has a different set of Makefiles, that use a different set of variables
- Though it is common to use `make` standard variables:
  `CC` (C compiler path), `CXX` (C++ compiler path), `LD` (linker path), `CFLAGS` (C compiler flags), `CXXFLAGS` (C++ compiler flags), `LDFLAGS` (linker flags)
- `DESTDIR` for installation destination, sometimes `PREFIX` for execution location
- Common sequence

```
$ make CC=arm-linux-gcc \
       CFLAGS=-I/path/to/headers \
       LDFLAGS=-L/path/to/libraries
$ make DESTDIR=/installation/path install
```

- Need to read the documentation (if any), read the Makefiles, and adapt to their behavior.



Example: *uftp* native compilation

Download and extract

```
$ wget http://sourceforge.net/projects/uftp-multicast/\
       files/source-tar/uftp-5.0.tar.gz
$ tar xf uftp-5.0.tar.gz
$ cd uftp-5.0
```

Build and install

```
$ make
cc  -g -Wall -Wextra [...]  -c server_announce.c
[...]
cc  -g -Wall -Wextra -o uftp uftp_common.o encrypt_openssl.o [...] \
   -lm -lcrypto -lpthread
$ make DESTDIR=/tmp/test install
```

Look at installed files

```
$ tree /tmp/test
/tmp/test/
└── usr
    ├── bin
    │   ├── uftp
    │   ├── uftpd
    │   ├── [...]
    └── share
        └── man
            └── man1
                ├── uftp.1
                ├── [...]

$ file /tmp/test/usr/bin/uftp
/tmp/test/usr/bin/uftp: ELF 64-bit LSB
executable, x86-64
```



Example: *uftp* cross-compilation

First attempt

```
$ export PATH=/xtools/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin:$PATH
$ make CC=arm-none-linux-gnueabihf-gcc
[...]
encryption.h:87:10: fatal error: openssl/rsa.h: No such file or directory
```

- Build fails because *uftp* uses *OpenSSL*
- This is an optional dependency that can be disabled using the special `make` variable `NO_ENCRYPTION`

Second attempt

```
$ make CC=arm-none-linux-gnueabihf-gcc NO_ENCRYPTION=1
arm-none-linux-gnueabihf-gcc  -g -Wall -Wextra [...]  -c server_announce.c
[...]
arm-none-linux-gnueabihf-gcc  -g -Wall -Wextra -o uftp uftp_common.o encrypt_none.o [...] -lm -lpthread
$ make DESTDIR=/tmp/target NO_ENCRYPTION=1 install
$ file /tmp/target/usr/bin/uftp
/tmp/target/usr/bin/uftp: ELF 32-bit LSB executable, ARM
```



Example: *OpenSSL* cross-compilation

OpenSSL has a hand-written `Configure` shell script that needs to be invoked before the build.

Download/extract

```
$ wget https://www.openssl.org/source/openssl-1.1.1q.tar.gz
$ tar xf openssl-1.1.1q.tar.gz
$ cd openssl-1.1.1q
```

Configuration/build

```
$ CC=arm-none-linux-gnueabihf-gcc ./Configure --prefix=/usr \
    linux-generic32 no-asm
$ make
$ make DESTDIR=/tmp/staging install
```

Installed files

```
$ tree /tmp/staging
└── usr
    ├── bin
    │   └── openssl
    ├── include
    │   ├── openssl
    │   │   ├── rsa.h
    │   │   └── [...]
    ├── lib
    │   ├── libcrypto.a
    │   ├── libcrypto.so -> libcrypto.so.1.1
    │   ├── libcrypto.so.1.1
    │   ├── [...]
    │   └── pkgconfig
    │       ├── libcrypto.pc
    │       └── [...]
    └── share
        ├── doc
        │   └── openssl
        └── man
```



Example: *uftp* with *OpenSSL* support

```
$ make CC=arm-none-linux-gnueabihf-gcc
encryption.h:87:10: fatal error: openssl/rsa.h:
   No such file or directory
[...]
```

It cannot find the header, let's add `CFLAGS` pointing to where OpenSSL headers are installed.

```
$ make CC=arm-none-linux-gnueabihf-gcc
       CFLAGS=-I/tmp/staging/usr/include
[... build OK, but at link time ...]
ld: cannot find -lcrypto
```

Compilation of object files work, but link fails as the linker cannot find the OpenSSL library. Let's add `LDFLAGS` pointing to where the OpenSSL libraries are installed.

```
$ make CC=arm-none-linux-gnueabihf-gcc
       CFLAGS=-I/tmp/staging/usr/include
       LDFLAGS=-L/tmp/staging/usr/lib
[... builds OK! ...]
$ make DESTDIR=/tmp/target install
```

Now it builds and installs fine!

```
$ arm-none-linux-gnueabihf-readelf -d /tmp/target/usr/bin/uftp
[...]
 0x00000001 (NEEDED) Shared library: [libm.so.6]
 0x00000001 (NEEDED) Shared library: [libcrypto.so.1.1]
 0x00000001 (NEEDED) Shared library: [libpthread.so.0]
 0x00000001 (NEEDED) Shared library: [libc.so.6]
[...]
```

We can indeed see that `uftp` is linked against the `libcrypto.so.1.1` shared library.

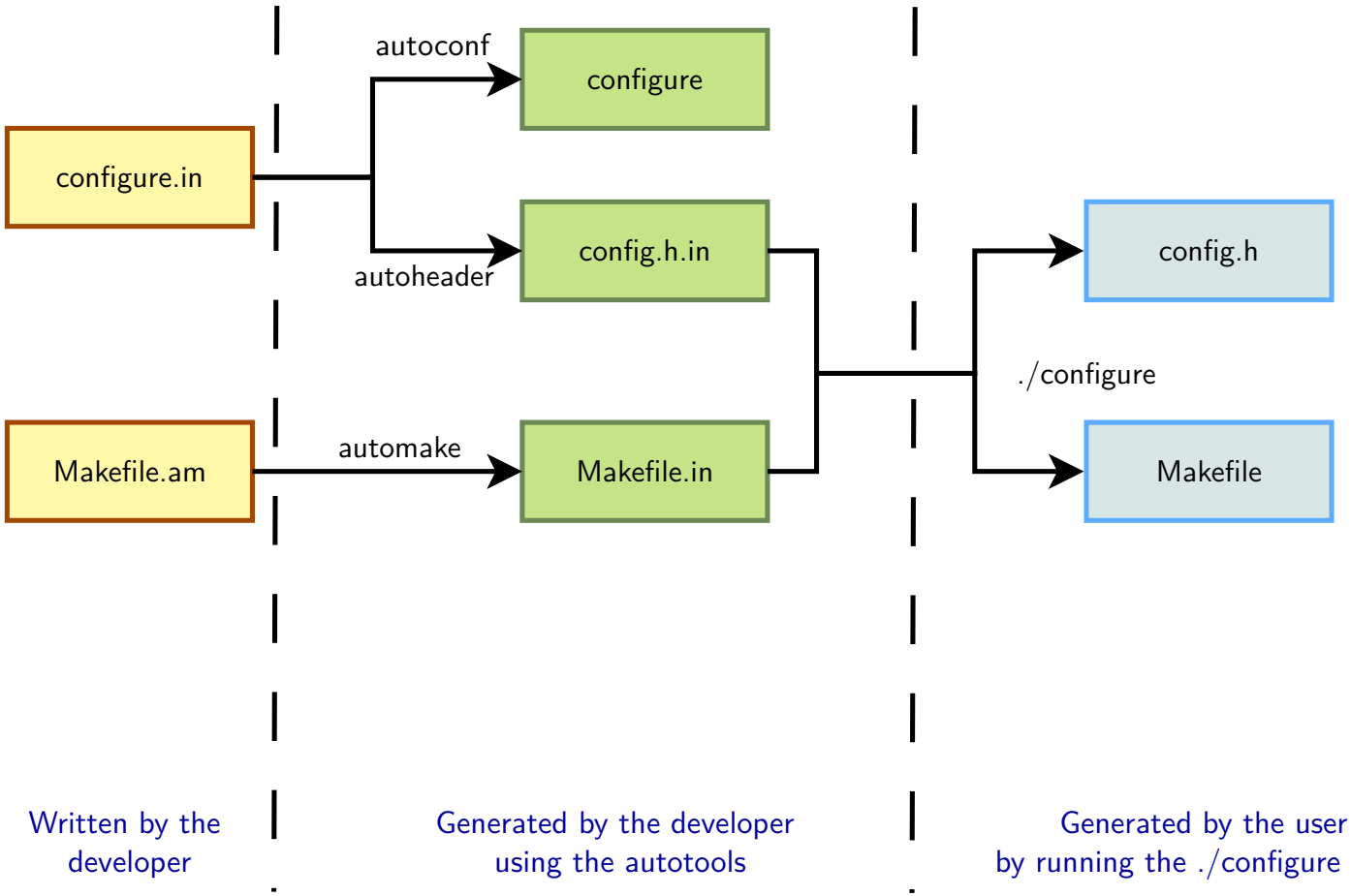


Autotools

- A family of tools, which associated together form a complete and extensible build system
  - **autoconf** is used to handle the configuration of the software package
  - **automake** is used to generate the Makefiles needed to build the software package
  - **libtool** is used to handle the generation of shared libraries in a system-independent way
- Most of these tools are old and relatively complicated to use
- But they are used by a large number of software components, even though *Meson* is gaining significant traction as a replacement today
- See also Bootlin Autotools training materials



automake / autoconf / autoheader





automake / autoconf

- Files written by the developer
  - `configure.in` describes the configuration options and the checks done at configure time
  - `Makefile.am` describes how the software should be built
- The `configure` script and the `Makefile.in` files are generated by `autoconf` and `automake` respectively.
  - They should never be modified directly
  - Software downloaded as a tarball: usually shipped pre-generated in the tarball
  - Software downloaded from Git: no pre-generated files under version control, so they must be generated
- The `Makefile` files are generated at configure time, before compiling
  - They are never shipped in the software package.



autotools usage: four steps

1. Only if needed: generate `configure` and `Makefile.in`. Either using *autoreconf* tool, or sometimes an `autogen.sh` script is provided by the package
2. **Configuration:** `./configure`
   - `./configure --help` is very useful
   - `--prefix`: execution location
   - `--host`: target machine when cross-compiling, if not provided, auto-detected. Also used as the cross-compiler prefix.
   - Often `--enable-<foo>`, `--disable-<foo>`, `--with-<foo>`, `--without-<foo>` for optional features.
   - `CC`, `CXX`, `CFLAGS`, `CXXFLAGS`, `LDFLAGS` and many more variables
3. **Build:** `make`
4. **Installation:** `make install`
   - `DESTDIR` variable for *diverted installation*



Example: can-utils native compilation

Download

```
$ git clone https://github.com/linux-can/can-utils.git
$ cd can-utils/
$ git checkout v2021.08.0
$ ls -1 configure* *makefile*
configure.ac GNUmakefile.am
```

No `configure` and `GNUmakefile.in`, *autoreconf needed*.

Autoreconf

```
$ autoreconf -i
$ ls -1 configure* *makefile*
configure configure.ac GNUmakefile.am GNUmakefile.in
```

Configuration

```
$ ./configure --prefix=/usr
$ ls -1 *makefile*
GNUmakefile GNUmakefile.am GNUmakefile.in
```

We now have the `GNUmakefile`, we can build and install.

Build/install

```
$ make
$ make DESTDIR=/tmp/test install
$ file /tmp/test/usr/bin/candump
/tmp/test/usr/bin/candump: ELF 64-bit LSB executable,
x86-64
```



Example: *can-utils* cross-compilation

```
$ export PATH=/xtools/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin:$PATH
$ ./configure --prefix=/usr --host=arm-none-linux-gnueabihf
$ make
$ make DESTDIR=/tmp/target install
$ file /tmp/target/usr/bin/candump
/tmp/target/usr/bin/candump: ELF 32-bit LSB executable, ARM
```

Note: This is a simple example, as *can-utils* does not have any dependency other than the C library, and has a simple `configure.ac` file.



CMake https://en.wikipedia.org/wiki/CMake

- More modern build system, started in 1999, maintained by a company called *Kitware*
- Used by Qt 6, KDE, and many projects which didn't like *autotools*
- Perhaps losing traction these days in favor of *Meson*
- Needs `cmake` installed on your machine
- Based on:
  - `CMakeLists.txt` files that describe what the dependencies are and what to build and install
  - `cmake`, a tool that processes `CMakeLists.txt` to generate either Makefiles (default) or Ninja files (covered later)
- Typical sequence, when using the *Makefile* backend:
  1. `cmake .`
  2. `make`
  3. `make install`



Example: *cJSON* native compilation

Download

```
$ git clone https://github.com/DaveGamble/cJSON.git
$ cd cJSON
$ git checkout v1.7.15
```

Configure, build, install

```
$ cmake -DCMAKE_INSTALL_PREFIX=/usr .
$ make
$ make DESTDIR=/tmp/test install
```

Installed files

```
$ tree /tmp/test
/tmp/test/
└── usr
    ├── include
    │   └── cjson
    │       └── cJSON.h
    └── lib64
        ├── cmake
        │   └── cJSON
        │       ├── cjson.cmake
        │       ├── cJSONConfig.cmake
        │       ├── cJSONConfigVersion.cmake
        │       └── cjson-noconfig.cmake
        ├── libcjson.so -> libcjson.so.1
        ├── libcjson.so.1 -> libcjson.so.1.7.15
        ├── libcjson.so.1.7.15
        └── pkgconfig
            └── libcjson.pc
```



Example: *cJSON cross-compilation*

cJSON has no dependency on any other library, so cross-compiling it is very easy as only the C cross-compiler needs to be specified:

```
$ cmake -DCMAKE_INSTALL_PREFIX=/usr -DCMAKE_C_COMPILER=arm-none-linux-gnueabihf-gcc .
$ make
$ make DESTDIR=/tmp/target install
$ file /tmp/target/usr/lib/libcjson.so.1.7.15
/tmp/target/usr/lib/libcjson.so.1.7.15: ELF 32-bit LSB shared object, ARM
```



CMake *toolchain file*

- When cross-compiling with *CMake*, the number of arguments to pass to specify the paths to all cross-compiler tools, libraries, headers, and flags can become quite long.
- They can be grouped into a *toolchain file*, which defines *CMake* variables
- Can then be used with
  `cmake -DCMAKE_TOOLCHAIN_FILE=/path/to/toolchain-file.txt`
- Such a *toolchain file* is commonly provided by embedded Linux build systems: Buildroot, Yocto, etc.
- Facilitates cross-compilation using CMake
- https://cmake.org/cmake/help/latest/manual/cmake-toolchains.7.html



Meson

https://en.wikipedia.org/wiki/Meson_(software)

- The most modern one, written in Python
- Gaining big traction in lots of major open-source projects
- Processes `meson.build` + `meson_options.txt` and generates *Ninja* files
- *Ninja* is an alternative to `make`, with much shorter build times
- Needs `meson` and `ninja` installed on your machine
- Meson requires an *out-of-tree* build: the build directory must be distinct from the source directory
  1. `mkdir build`
  2. `cd build`
  3. `meson ..`
  4. `ninja`
  5. `ninja install`



Example: *ipcalc* native compilation

Download

```
$ git clone https://gitlab.com/ipcalc/ipcalc.git
$ cd ipcalc
$ git checkout 1.0.1
```

Configuration, build, installation

```
$ mkdir build
$ cd build
$ meson --prefix /usr ..
$ ninja
$ DESTDIR=/tmp/test ninja install
```

Installed files

```
$ tree /tmp/test
/tmp/test/
└── usr
    └── bin
        └── ipcalc
```



Meson *cross file*

- In a similar manner to CMake's *toolchain file*, *Meson* has a concept of *cross file*
- Small text file that contains variable definitions telling *Meson* all details needed for cross-compilation
- Can be created manually, or may be provided by an embedded Linux build systems such as Buildroot or Yocto.
- `--cross-file` option of *Meson*

Cross file example

```
[binaries]
c = 'arm-none-linux-gnueabihf-gcc'
strip = 'arm-none-linux-gnueabihf-strip'

[host_machine]
system = 'linux'
cpu_family = 'arm'
cpu = 'cortex-a9'
endian = 'little'
```



Example: *ipcalc* cross-compilation

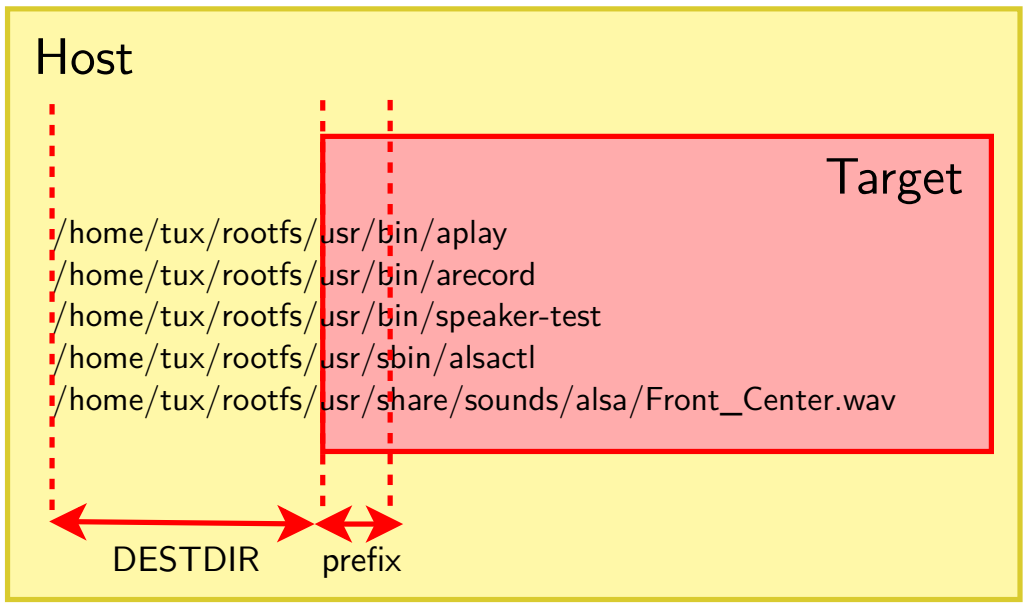
```
$ cat cross-file.txt
[binaries]
c = 'arm-none-linux-gnueabihf-gcc'
strip = 'arm-none-linux-gnueabihf-strip'

[host_machine]
system = 'linux'
cpu_family = 'arm'
cpu = 'cortex-a9'
endian = 'little'
$ mkdir build-cross
$ cd build-cross
$ meson --cross-file ../cross-file.txt --prefix /usr ..
$ ninja
$ DESTDIR=/tmp/target ninja install
$ file /tmp/target/usr/bin/ipcalc
/tmp/target/usr/bin/ipcalc: ELF 32-bit LSB executable, ARM
```



Distinction between *prefix* and *DESTDIR*

- There is often a confusion between *prefix* and *DESTDIR*
- Distinction is very important in cross-compilation context
- `prefix`: where the software will be executed from on the target
- `DESTDIR`: where the software is installed by the build system installation procedure. Allows to install in a different place than `prefix`, when creating a root filesystem for a different machine.





pkg-config

- `pkg-config` is a tool that allows to query a small database to get information on how to compile programs that depend on libraries
- https://people.freedesktop.org/~dbn/pkg-config-guide.html
- The database is made of `.pc` files, installed by default in `<prefix>/lib/pkgconfig/`.
- `pkg-config` is often used by *autotools*, *CMake*, *Meson* to find libraries
- By default, `pkg-config` looks in `/usr/lib/pkgconfig` for the `*.pc` files, and assumes that the paths in these files are correct.
- `PKG_CONFIG_LIBDIR` allows to set another location for the `*.pc` files.
- `PKG_CONFIG_SYSROOT_DIR` allows to prepend a directory to the paths mentioned in the `.pc` files and appearing in the `pkg-config` output.



pkg-config example for native compilation

```
$ pkg-config --list-all
openssl                         OpenSSL - Secure Sockets Layer and cryptography libraries and tools
zlib                            zlib - zlib compression library
blkid                           blkid - Block device id library
cairo-script                    cairo-script - script surface backend for cairo graphics library
cairo-pdf                       cairo-pdf - PDF surface backend for cairo graphics library
xcb-xinput                      XCB XInput - XCB XInput Extension (EXPERIMENTAL)
libcurl                         libcurl - Library to transfer files with ftp, http, etc.
[...]
$ pkg-config --cflags --libs openssl
-lssl -lcrypto
$ pkg-config --cflags --libs cairo-script
-I/usr/include/cairo -I/usr/include/libpng16 -I/usr/include/freetype2 -I/usr/include/harfbuzz
[...] -lcairo -lz
```



pkg-config example for cross-compilation

Use `PKG_CONFIG_LIBDIR`

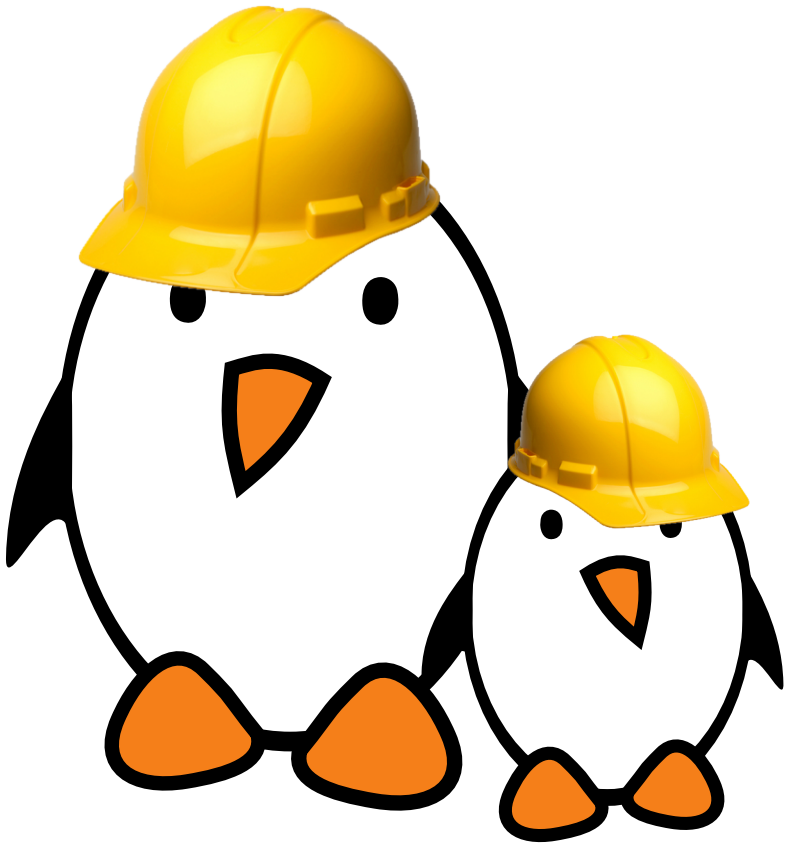
```
$ export PKG_CONFIG_LIBDIR=/tmp/staging/usr/lib/pkgconfig
$ pkg-config --list-all
openssl                                 OpenSSL - Secure Sockets Layer and cryptography
libraries and tools libssl              OpenSSL-libssl - Secure Sockets Layer and cryptography
libraries libcrypto                     OpenSSL-libcrypto - OpenSSL cryptography library
$ pkg-config --cflags --libs openssl
-I/usr/include -L/usr/lib -lssl -lcrypto
```

The `-L/usr/lib` is incorrect, we need to use `PKG_CONFIG_SYSROOT_DIR`.

Use `PKG_CONFIG_SYSROOT_DIR`

```
$ export PKG_CONFIG_SYSROOT_DIR=/tmp/staging/
$ pkg-config --cflags --libs openssl
-I/tmp/staging/usr/include -L/tmp/staging/usr/lib -lssl -lcrypto
```

Practical lab - Cross-compiling applications and libraries



Time to start the practical lab!

- Manual cross-compilation of several open-source libraries and applications for an embedded platform.
- Learning about common pitfalls and issues, and their solutions.
- This includes compiling *alsa-utils* package, and using its `speaker-test` program to test that audio works on the target.